

module-07-regex

Chapter 7: Pattern Matching with Regular Expressions | 46 programs in learning order

Source: Automate the Boring Stuff with Python 1e, Al Sweigart (CC-BY-NC-SA). Syllabus Ch7: re.compile, groups, pipe, ? * + {}, greedy, findall, classes, ^\$, dot-star, flags, sub

How to use this guide (beginner path)

1) Read Concepts once. 2) Study the Visuals - redraw each on paper. 3) Follow the Program map. 4) Work programs 01..N in order. [shell]: type every plain line into IDLE, compare with # OUT: comments (errors shown are intentional demos). [script]: save, predict output, run with F5. 5) Drill Interview prep. 6) Teach it: explain each visual aloud without looking.

Concepts in detail

Regex finds PATTERNS in text, not exact words. Compile once (phoneReg = re.compile(r'\d\d\d-\d\d\d-\d\d\d\d')), then search(). Parentheses make groups: mo.group(1). Pipe | means or; ? optional; * zero-or-more; + one-or-more; {3} / {3,5} exact counts. .* is greedy (longest match) unless you add ? for nongreedy. ^ anchors start, \$ end. findall() returns all hits. IGNORECASE/DOTALL/VERBOSE flags + sub() for replacement round it out.

Key ideas to carry forward

- \d digit, \w word char, \s space; [aeiou] custom class; [^...] negated class.
- Bat(wo)?man matches Batman and Batwoman; Bat\w+ needs at least one word char.
- Greedy (.*?) vs nongreedy (.*?) - the phone/email extractor project lives or dies on this.
- re.VERBOSE lets a complex pattern span commented lines.

Classic beginner mistakes

- ! Forgetting raw strings so \d becomes an escape bug - always r'...'.
- ! match() only checks the start; use search() for anywhere.
- ! Greedy .* swallowing two intended matches into one.

Study order hint: 01-04 why-regex motivation, 05-23 core operators in order, 24-41 classes/anchors/flags/sub, 42-46 the extractor project.

Interview prep - questions you must answer cold

Say each answer aloud in under 60 seconds, then prove it with one program from this module.

Q1. Why regex over plain string code?

A: One pattern handles all variants (phone formats) where str methods need nested conditions. Compile once, reuse everywhere.

Q2. Explain groups and group().

A: Parentheses capture parts: `(\d\d\d)-(\d+)` - `group(0)` whole match, `group(1..n)` each part for reformatting.

Q3. ? vs * vs + vs {m,n}?

A: ? optional (0-1); * zero-or-more; + one-or-more; {3} exact, {2,5} range. Greedy by default.

Q4. Greedy vs non-greedy?

A: `'<.*>'` on `'<a><b>'` grabs everything; `'<.*?>'` grabs `'<a>'`. Critical for scraping/extraction.

Q5. findall vs search?

A: `search` returns first match object; `findall` returns all matches (tuples when groups exist).

Q6. Flags and sub()?

A: `IGNORECASE`, `DOTALL` (* matches newlines), `VERBOSE` (readable patterns); `sub(repl, text)` replaces matches.

Teach it to others

The one analogy that sticks

Regex is a wanted poster: `\d` means 'any digit suspect', `{3}` means 'exactly three of them', groups are circled faces to identify later. Greedy is a net that scoops the whole pond; nongreedy (`?`) scoops one fish.

What your students will get wrong

- x Patterns work without raw strings (backslashes corrupt - always `r'...'`).
- x `match()` searches anywhere (only start - use `search`).
- x `.*` is safe by default (it over-matches; prefer `.*?` or char classes).

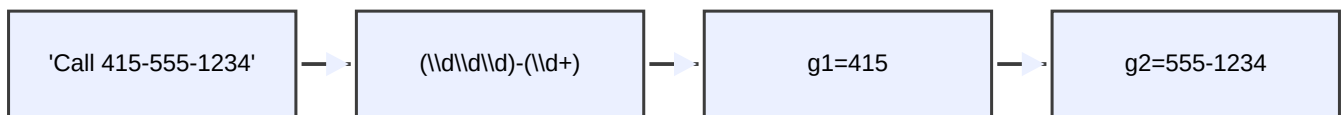
5-minute whiteboard plan

1. Evolve 415-555-1234 matcher from literal to `\d` groups.
2. Show greedy swallowing two tags, then fix with `?`.
3. Students extract all emails from a pasted paragraph.
4. Exit ticket: write `^{\d{5}}$` and say what it validates.

Visualize it - draw these from memory

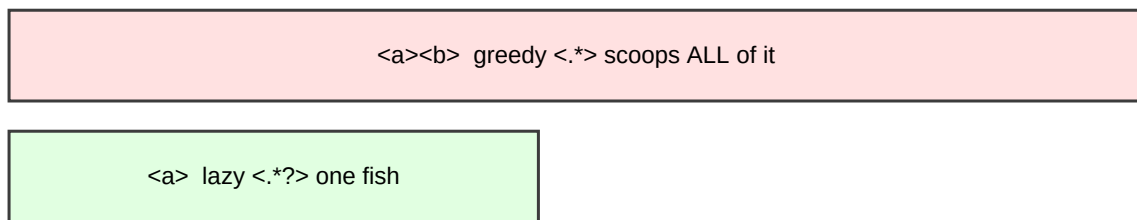
Redraw each figure on paper with labels before you touch code. If you cannot draw it, you cannot teach it.

Visual 1 - pattern pipeline: text in, groups out



Parentheses circle the suspects (groups) so you can identify them later with `group(1)`, `group(2)`.

Visual 2 - greedy net vs lazy hook



`.*` grabs the longest possible match. Add `?` to take the shortest. Always use `r'...'` raw strings.

More examples - same idea, new clothes

Three bonus drills per module. Cover the output, predict, then run.

X1. Hashtag harvester

```
import re
text = 'Go #py and #regex!'
print(re.findall(r'#\w+', text))
# EXPECTED: ['#py', '#regex']
```

X2. Mask card numbers

```
import re
s = 'Card 4111-1111-1111-1111 ok'
print(re.sub(r'\d{4}-\d{4}-\d{4}-\d{4}', '****-****-****-****', s))
# EXPECTED: Card ****-****-****-**** ok
```

X3. Greedy vs lazy

```
import re
s = '<a><b>'
print(re.search(r'<.*>', s).group())
print(re.search(r'<.*?>', s).group())
# EXPECTED: <a><b> / <a>
```

Program map - all 46 at a glance

```
01 01_finding_patterns_text.py
02 02_finding_patterns_text_2.py
03 03_finding_patterns_text_message.py
04 04_finding_patterns_text_phone.py
05 05_creating_regex_objects.py
06 06_creating_regex_objects_phonenumrege.py
07 07_matching_regex_objects.py
08 08_grouping_parentheses.py
09 09_grouping_parentheses_mo.py
10 10_grouping_parentheses_phonenumrege.py
11 11_matching_multiple_groups.py
12 12_matching_multiple_groups_batregex.py
13 13_optional_matching_question.py
14 14_optional_matching_question_phoneregex.py
15 15_matching_zero_or.py
16 16_matching_one_or.py
17 17_matching_specific_repetitions.py
18 18_matching_specific_repetitions_ha.py
19 19_matching_specific_repetitions_haregex.py
20 20_greedy_nongreedy_matching.py
21 21_findall_method.py
22 22_findall_method_phonenumrege.py
23 23_findall_method_phonenumrege_2.py
24 24_character_classes.py
25 25_making_your_own.py
26 26_making_your_own_consonantreg.py
27 27_caret_dollar_sign.py
28 28_caret_dollar_sign_endswithnumb.py
29 29_caret_dollar_sign_wholestringi.py
30 30_wildcard_character.py
31 31_matching_everything_dot.py
32 32_matching_everything_dot_nongreedyreg.py
33 33_matching_newlines_dot.py
34 34_case_insensitive_matching.py
35 35_case_insensitive_matching_robocop.py
36 36_substituting_strings_sub.py
37 37_substituting_strings_sub_agentnamesre.py
```

38 38_managing_complex_regexes.py
39 39_managing_complex_regexes_phoneregex.py
40 40_combining_re_ignorecase.py
41 41_combining_re_ignorecase_someregexval.py
42 42_step_1_create.py
43 43_step_2_create.py
44 44_step_3_find.py
45 45_step_4_join.py
46 46_running_program.py

Topic: Finding Patterns of Text Without Regular Expressions

01/46 01_finding_patterns_text.py [script]

Script: save and run; predict the output first, then compare.

```
def isPhoneNumber(text):
    if len(text) != 12:
        return False
    for i in range(0, 3):
        if not text[i].isdecimal():
            return False
    if text[3] != '-':
        return False
    for i in range(4, 7):
        if not text[i].isdecimal():
            return False
    if text[7] != '-':
        return False
    for i in range(8, 12):
        if not text[i].isdecimal():
            return False
    return True

print('415-555-4242 is a phone number:')
print(isPhoneNumber('415-555-4242'))
print('Moshi moshi is a phone number:')
print(isPhoneNumber('Moshi moshi'))
```

02/46 02_finding_patterns_text_2.py [script]

Script: save and run; predict the output first, then compare.

```
415-555-4242 is a phone number:  
True  
Moshi moshi is a phone number:  
False
```

03/46 03_finding_patterns_text_message.py [script]

Script: save and run; predict the output first, then compare.

```
message = 'Call me at 415-555-1011 tomorrow. 415-555-9999 is my office.'  
for i in range(len(message)):  
    chunk = message[i:i+12]  
    if isPhoneNumber(chunk):  
        print('Phone number found: ' + chunk)  
print('Done')
```

04/46 04_finding_patterns_text_phone.py [script]

Script: save and run; predict the output first, then compare.

```
Phone number found: 415-555-1011
```

```
Phone number found: 415-555-9999
```

```
Done
```

Topic: Creating Regex Objects

05/46 05_creating_regex_objects.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
import re
```

06/46 06_creating_regex_objects_phonenumrege.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
phoneNumRegex = re.compile(r'\d\d\d-\d\d\d-\d\d\d\d')
```

Topic: Matching Regex Objects

07/46 07_matching_regex_objects.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
phoneNumRegex = re.compile(r'\d\d\d-\d\d\d-\d\d\d\d')
mo = phoneNumRegex.search('My number is 415-555-4242.')
print('Phone number found: ' + mo.group())
# OUT: Phone number found: 415-555-4242
```

Topic: Grouping with Parentheses

08/46 08_grouping_parentheses.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
phoneNumRegex = re.compile(r'(\d\d\d)-(\d\d\d-\d\d\d\d)')  
mo = phoneNumRegex.search('My number is 415-555-4242.')  
mo.group(1)  
# OUT: '415'  
mo.group(2)  
# OUT: '555-4242'  
mo.group(0)  
# OUT: '415-555-4242'  
mo.group()  
# OUT: '415-555-4242'
```

09/46 09_grouping_parentheses_mo.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
mo.groups()  
# OUT: ('415', '555-4242')  
areaCode, mainNumber = mo.groups()  
print(areaCode)  
# OUT: 415  
print(mainNumber)  
# OUT: 555-4242
```

10/46 10_grouping_parentheses_phonenumrege.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
phoneNumRegex = re.compile(r'(\d\d\d\) (\d\d\d-\d\d\d\d)')  
mo = phoneNumRegex.search('My phone number is (415) 555-4242.')  
mo.group(1)  
# OUT: '(415)'  
mo.group(2)  
# OUT: '555-4242'
```

Topic: Matching Multiple Groups with the Pipe

11/46 11_matching_multiple_groups.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
heroRegex = re.compile (r'Batman|Tina Fey')  
mo1 = heroRegex.search('Batman and Tina Fey.')  
mo1.group()  
# OUT: 'Batman'  
  
mo2 = heroRegex.search('Tina Fey and Batman.')  
mo2.group()  
# OUT: 'Tina Fey'
```

12/46 12_matching_multiple_groups_batregex.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
batRegex = re.compile(r'Bat(man|mobile|copter|bat)')  
mo = batRegex.search('Batmobile lost a wheel')  
mo.group()  
# OUT: 'Batmobile'  
mo.group(1)  
# OUT: 'mobile'
```

Topic: Optional Matching with the Question Mark

13/46 13_optional_matching_question.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
batRegex = re.compile(r'Bat(wo)?man')  
mo1 = batRegex.search('The Adventures of Batman')  
mo1.group()  
# OUT: 'Batman'  
  
mo2 = batRegex.search('The Adventures of Batwoman')  
mo2.group()  
# OUT: 'Batwoman'
```

14/46 14_optional_matching_question_phoneregex.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
phoneRegex = re.compile(r'(\d\d\d-)?\d\d\d-\d\d\d\d')  
mo1 = phoneRegex.search('My number is 415-555-4242')  
mo1.group()  
# OUT: '415-555-4242'  
  
mo2 = phoneRegex.search('My number is 555-4242')  
mo2.group()  
# OUT: '555-4242'
```

Topic: Matching Zero or More with the Star

15/46 15_matching_zero_or.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
batRegex = re.compile(r'Bat(wo)*man')
```

```
mo1 = batRegex.search('The Adventures of Batman')
```

```
mo1.group()
```

```
# OUT: 'Batman'
```

```
mo2 = batRegex.search('The Adventures of Batwoman')
```

```
mo2.group()
```

```
# OUT: 'Batwoman'
```

```
mo3 = batRegex.search('The Adventures of Batwowowowoman')
```

```
mo3.group()
```

```
# OUT: 'Batwowowowoman'
```

Topic: Matching One or More with the Plus

16/46 16_matching_one_or.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).
```

```
batRegex = re.compile(r'Bat(wo)+man')  
mo1 = batRegex.search('The Adventures of Batwoman')  
mo1.group()  
# OUT: 'Batwoman'  
  
mo2 = batRegex.search('The Adventures of Batwowowowoman')  
mo2.group()  
# OUT: 'Batwowowowoman'  
  
mo3 = batRegex.search('The Adventures of Batman')  
mo3 == None  
# OUT: True
```

Topic: Matching Specific Repetitions with Curly Brackets

17/46 17_matching_specific_repetitions.py [script]

Script: save and run; predict the output first, then compare.

```
(Ha){3}
```

```
(Ha)(Ha)(Ha)
```

18/46 18_matching_specific_repetitions_ha.py [script]

Script: save and run; predict the output first, then compare.

```
(Ha){3,5}
```

```
((Ha)(Ha)(Ha))|((Ha)(Ha)(Ha)(Ha))|((Ha)(Ha)(Ha)(Ha)(Ha))
```

19/46 19_matching_specific_repetitions_haregex.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
haRegex = re.compile(r'(Ha){3}')  
mo1 = haRegex.search('HaHaHa')  
mo1.group()  
# OUT: 'HaHaHa'  
  
mo2 = haRegex.search('Ha')  
mo2 == None  
# OUT: True
```

Topic: Greedy and Nongreedy Matching

20/46 20_greedy_nongreedy_matching.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.
# Lines starting with >>> are input; other lines are expected output (commented).

greedyHaRegex = re.compile(r'(Ha){3,5}')
mo1 = greedyHaRegex.search('HaHaHaHaHa')
mo1.group()
# OUT: 'HaHaHaHaHa'

nongreedyHaRegex = re.compile(r'(Ha){3,5}?')
mo2 = nongreedyHaRegex.search('HaHaHaHaHa')
mo2.group()
# OUT: 'HaHaHa'
```

Topic: The findall() Method

21/46 21_findall_method.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
phoneNumRegex = re.compile(r'\d\d\d-\d\d\d-\d\d\d\d')
mo = phoneNumRegex.search('Cell: 415-555-9999 Work: 212-555-0000')
mo.group()
# OUT: '415-555-9999'
```

22/46 22_findall_method_phonenumrege.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
phoneNumRegex = re.compile(r'\d\d\d-\d\d\d-\d\d\d\d') # has no groups  
phoneNumRegex.findall('Cell: 415-555-9999 Work: 212-555-0000')  
# OUT: ['415-555-9999', '212-555-0000']
```

23/46 23_findall_method_phonenumrege_2.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
phoneNumRegex = re.compile(r'(\d\d\d)-(\d\d\d)-(\d\d\d\d)') # has groups  
phoneNumRegex.findall('Cell: 415-555-9999 Work: 212-555-0000')  
# OUT: [('415', '555', '9999'), ('212', '555', '0000')]
```

Topic: Character Classes

24/46 24_character_classes.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
xmasRegex = re.compile(r'\d+\s\w+')  
xmasRegex.findall('12 drummers, 11 pipers, 10 lords, 9 ladies, 8 maids, 7  
# OUT: swans, 6 geese, 5 rings, 4 birds, 3 hens, 2 doves, 1 partridge')  
# OUT: ['12 drummers', '11 pipers', '10 lords', '9 ladies', '8 maids', '7 swans', '6  
# OUT: geese', '5 rings', '4 birds', '3 hens', '2 doves', '1 partridge']
```

Topic: Making Your Own Character Classes

25/46 25_making_your_own.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
vowelRegex = re.compile(r'[aeiouAEIOU]')
```

```
vowelRegex.findall('Robocop eats baby food. BABY FOOD.')
```

```
# OUT: ['o', 'o', 'o', 'e', 'a', 'a', 'o', 'o', 'A', 'O', 'O']
```

26/46 26_making_your_own_consonantreg.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
consonantRegex = re.compile(r'^aeiouAEIOU')  
consonantRegex.findall('Robocop eats baby food. BABY FOOD.')  
# OUT: ['R', 'b', 'c', 'p', ' ', 't', 's', ' ', 'b', 'b', 'y', ' ', 'f', 'd', '.', ' '  
# OUT: ', 'B', 'B', 'Y', ' ', 'F', 'D', '.']
```

Topic: The Caret and Dollar Sign Characters

27/46 27_caret_dollar_sign.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
beginsWithHello = re.compile(r'^Hello')  
beginsWithHello.search('Hello world!')  
# OUT: <_sre.SRE_Match object; span=(0, 5), match='Hello'>  
beginsWithHello.search('He said hello.') == None  
# OUT: True
```

28/46 28_caret_dollar_sign_endswithnumb.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
endsWithNumber = re.compile(r'\d$')  
endsWithNumber.search('Your number is 42')  
# OUT: <_sre.SRE_Match object; span=(16, 17), match='2'>  
endsWithNumber.search('Your number is forty two.') == None  
# OUT: True
```

29/46 29_caret_dollar_sign_wholestringi.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
wholeStringIsNum = re.compile(r'^\d+$')  
wholeStringIsNum.search('1234567890')  
# OUT: <_sre.SRE_Match object; span=(0, 10), match='1234567890'>  
wholeStringIsNum.search('12345xyz67890') == None  
# OUT: True  
wholeStringIsNum.search('12 34567890') == None  
# OUT: True
```

Topic: The Wildcard Character

30/46 30_wildcard_character.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
atRegex = re.compile(r'.at')
```

```
atRegex.findall('The cat in the hat sat on the flat mat.')
```

```
# OUT: ['cat', 'hat', 'sat', 'lat', 'mat']
```

Topic: Matching Everything with Dot-Star

31/46 31_matching_everything_dot.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
nameRegex = re.compile(r'First Name: (.*?) Last Name: (.*?)')  
mo = nameRegex.search('First Name: Al Last Name: Sweigart')  
mo.group(1)  
# OUT: 'Al'  
mo.group(2)  
# OUT: 'Sweigart'
```

32/46 32_matching_everything_dot_nongreedyreg.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
>>> nongreedyRegex = re.compile(r'<.*?>')  
>>> mo = nongreedyRegex.search('<To serve man> for dinner.>')  
>>> mo.group()  
# OUT: '<To serve man>'  
  
>>> greedyRegex = re.compile(r'<.*>')  
>>> mo = greedyRegex.search('<To serve man> for dinner.>')  
>>> mo.group()  
# OUT: '<To serve man> for dinner.>'
```

Topic: Matching Newlines with the Dot Character

33/46 33_matching_newlines_dot.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.
# Lines starting with >>> are input; other lines are expected output (commented).

noNewlineRegex = re.compile('.')
noNewlineRegex.search('Serve the public trust.\nProtect the innocent.
# OUT: \nUphold the law.').group()
# OUT: 'Serve the public trust.'

newlineRegex = re.compile('.', re.DOTALL)
newlineRegex.search('Serve the public trust.\nProtect the innocent.
# OUT: \nUphold the law.').group()
# OUT: 'Serve the public trust.\nProtect the innocent.\nUphold the law.'
```

Topic: Case-Insensitive Matching

34/46 34_case_insensitive_matching.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
regex1 = re.compile('Robocop')
regex2 = re.compile('ROBOCOP')
regex3 = re.compile('robOcop')
regex4 = re.compile('RobocOp')
```

35/46 35_case_insensitive_matching_robocop.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
robocop = re.compile(r'robocop', re.I)  
robocop.search('Robocop is part man, part machine, all cop.').group()  
# OUT: 'Robocop'  
  
robocop.search('ROBOCOP protects the innocent.').group()  
# OUT: 'ROBOCOP'  
  
robocop.search('Al, why does your programming book talk about robocop so much?').group()  
# OUT: 'robocop'
```

Topic: Substituting Strings with the sub() Method

36/46 36_substituting_strings_sub.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
namesRegex = re.compile(r'Agent \w+')
```

```
namesRegex.sub('CENSORED', 'Agent Alice gave the secret documents to Agent Bob.')
```

```
# OUT: 'CENSORED gave the secret documents to CENSORED.'
```

37/46 37_substituting_strings_sub_agentnamesre.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).
```

```
agentNamesRegex = re.compile(r'Agent (\w)\w*')  
agentNamesRegex.sub(r'\1****', 'Agent Alice told Agent Carol that Agent  
# OUT: Eve knew Agent Bob was a double agent.')  
# OUT: A**** told C**** that E**** knew B**** was a double agent.'
```

Topic: Managing Complex Regexes

38/46 38_managing_complex_regexes.py [script]

Script: save and run; predict the output first, then compare.

```
phoneRegex = re.compile(r'((\d{3}|\d{3}\d{3})?(\s|-|\.)?\d{3})(\s|-|\.)\d{4}\s*(ext|x|ext.)\s*\d{2,5})?')
```

39/46 39_managing_complex_regexes_phoneregex.py [script]

Script: save and run; predict the output first, then compare.

```
phoneRegex = re.compile(r'''(
    (\d{3}|\(\d{3}\))?           # area code
    (\s|-|\.)?                  # separator
    \d{3}                        # first 3 digits
    (\s|-|\.)                   # separator
    \d{4}                        # last 4 digits
    (\s*(ext|x|ext.)\s*\d{2,5})? # extension
    )''', re.VERBOSE)
```

Topic: Combining re.IGNORECASE, re.DOTALL, and re.VERBOSE

40/46 40_combining_re_ignorecase.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
someRegexValue = re.compile('foo', re.IGNORECASE | re.DOTALL)
```

41/46 41_combining_re_ignorecase_someregexval.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.
```

```
# Lines starting with >>> are input; other lines are expected output (commented).
```

```
someRegexValue = re.compile('foo', re.IGNORECASE | re.DOTALL | re.VERBOSE)
```

Topic: Step 1: Create a Regex for Phone Numbers

42/46 42_step_1_create.py [script]

Script: save and run; predict the output first, then compare.

```
#!/ python3
# phoneAndEmail.py - Finds phone numbers and email addresses on the clipboard.

import pyperclip, re

phoneRegex = re.compile(r'''(
    (\d{3}|\(\d{3}\))?           # area code
    (\s|-|\.)?                  # separator
    (\d{3})                      # first 3 digits
    (\s|-|\.)?                  # separator
    (\d{4})                      # last 4 digits
    (\s*(ext|x|ext.)\s*(\d{2,5}))? # extension
    )''', re.VERBOSE)

# TODO: Create email regex.

# TODO: Find matches in clipboard text.

# TODO: Copy results to the clipboard.
```

Topic: Step 2: Create a Regex for Email Addresses

43/46 43_step_2_create.py [script]

Script: save and run; predict the output first, then compare.

```
#!/ python3
# phoneAndEmail.py - Finds phone numbers and email addresses on the clipboard.
import pyperclip, re

phoneRegex = re.compile(r'''
--snip--

# Create email regex.
emailRegex = re.compile(r'''(
    [a-zA-Z0-9._%+-]+      # username
    @                      # @ symbol
    [a-zA-Z0-9.-]+        # domain name
    (\.[a-zA-Z]{2,4})      # dot-something
    )''', re.VERBOSE)

# TODO: Find matches in clipboard text.

# TODO: Copy results to the clipboard.
```

Topic: Step 3: Find All Matches in the Clipboard Text

44/46 44_step_3_find.py [script]

Script: save and run; predict the output first, then compare.

```
#!/ python3
# phoneAndEmail.py - Finds phone numbers and email addresses on the clipboard.

import pyperclip, re

phoneRegex = re.compile(r'''(
    --snip--

# Find matches in clipboard text.
text = str(pyperclip.paste())
matches = []
for groups in phoneRegex.findall(text):
    phoneNum = '-'.join([groups[1], groups[3], groups[5]])
    if groups[8] != '':
        phoneNum += ' x' + groups[8]
    matches.append(phoneNum)
for groups in emailRegex.findall(text):
    matches.append(groups[0])

# TODO: Copy results to the clipboard.
```

Topic: Step 4: Join the Matches into a String for the Clipboard

45/46 45_step_4_join.py [script]

Script: save and run; predict the output first, then compare.

```
#!/ python3
# phoneAndEmail.py - Finds phone numbers and email addresses on the clipboard.

--snip--
for groups in emailRegex.findall(text):
    matches.append(groups[0])

# Copy results to the clipboard.
if len(matches) > 0:
    pyperclip.copy('\n'.join(matches))
    print('Copied to clipboard:')
    print('\n'.join(matches))
else:
    print('No phone numbers or email addresses found.')
```

Topic: Running the Program

46/46 46_running_program.py [script]

Script: save and run; predict the output first, then compare.

Copied to clipboard:

800-420-7240

415-863-9900

415-863-9950

info@nostarch.com

media@nostarch.com

academic@nostarch.com

help@nostarch.com